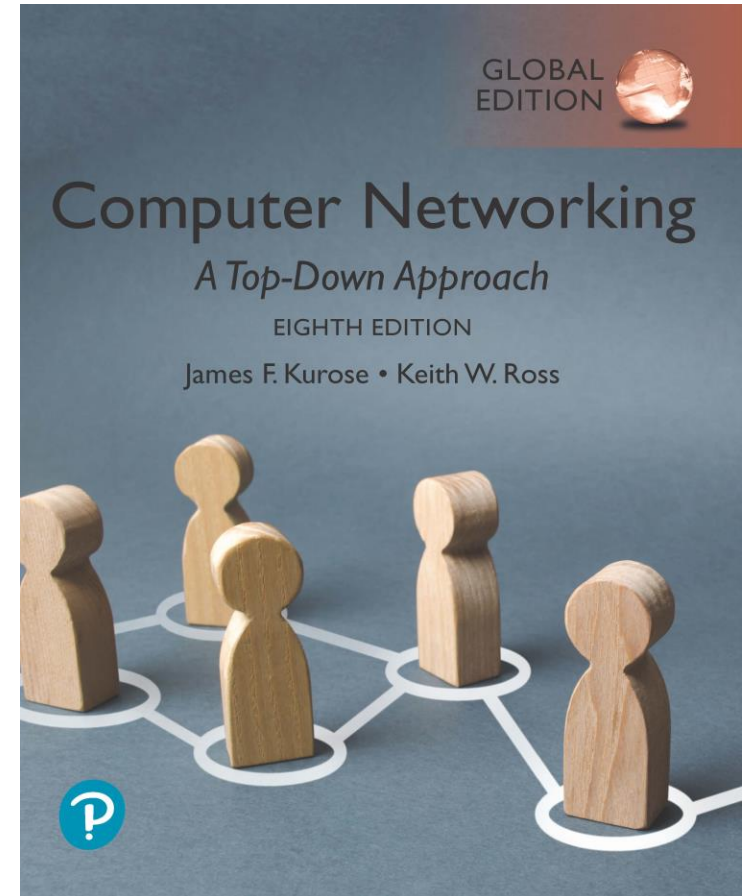


Chapter 3

Transport Layer



Computer Networking: A Top-Down Approach

8th Edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP
- Principles of reliable data transfer
- **Connection-oriented transport: TCP**
 - **segment structure**
 - **reliable data transfer**
 - flow control
 - connection management
- Principles of congestion control
- TCP congestion control



TCP: overview

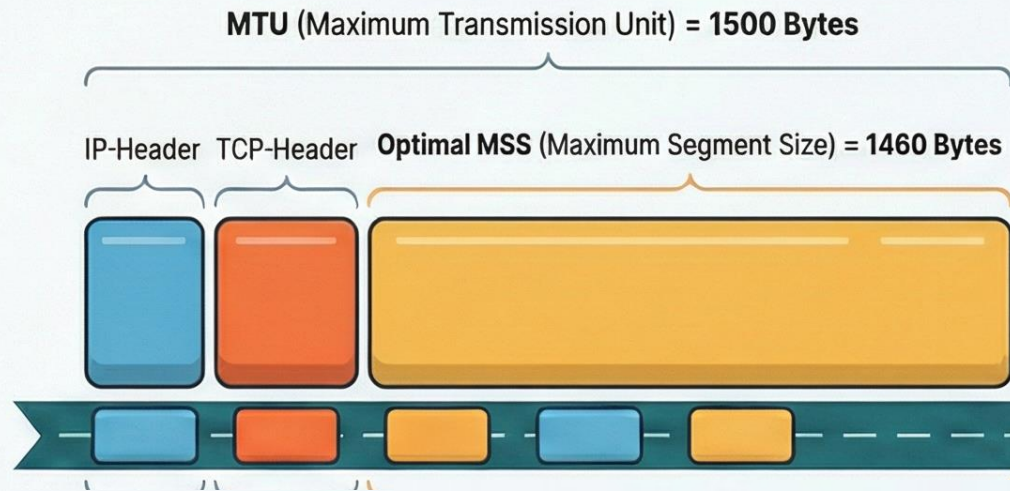
RFCs: 793, 1122, 2018, 5681, 7323

- point-to-point:
 - one sender, one receiver
- reliable, in-order *byte stream*:
 - no “message boundaries”
- full duplex data:
 - bi-directional data flow in same connection
 - MSS: maximum segment size
- cumulative ACKs
- pipelining:
 - TCP congestion and flow control set window size
- connection-oriented:
 - handshaking (exchange of control messages) initializes sender, receiver state before data exchange
- flow controlled:
 - sender will not overwhelm receiver

TCP MSS: Nøkkelen til effektiv dataflyt

Maksimer ytelse, unngå forsinkelser og fragmentering i nettverket.

Regnestykket og forhandlingen

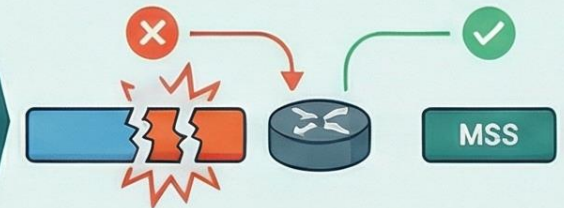


Forhandling under håndtrykket:
Enhetene blir enige om den laveste foreslåtte MSS-verdien for å sikre stabilitet.

Hvorfor MSS er avgjørende

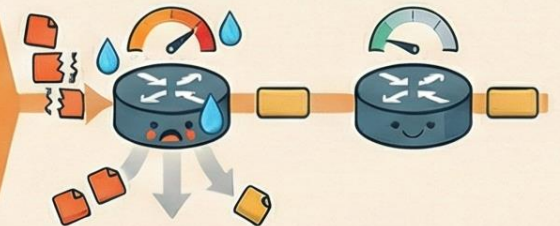
Unngå IP-fragmentering

MSS sikrer at pakker ikke blir delt opp av rutere undervegs i nettverket.



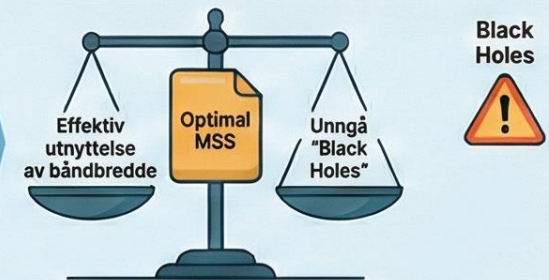
Redusert prosessorlast og tap

Uten fragmentering slipper rutere ekstra arbeid, og man minsker risikoen for tapte data.



Den gyldne middelvei

Riktig MSS balanserer effektiv utnyttelse av båndbredde mot faren for "Black Holes".

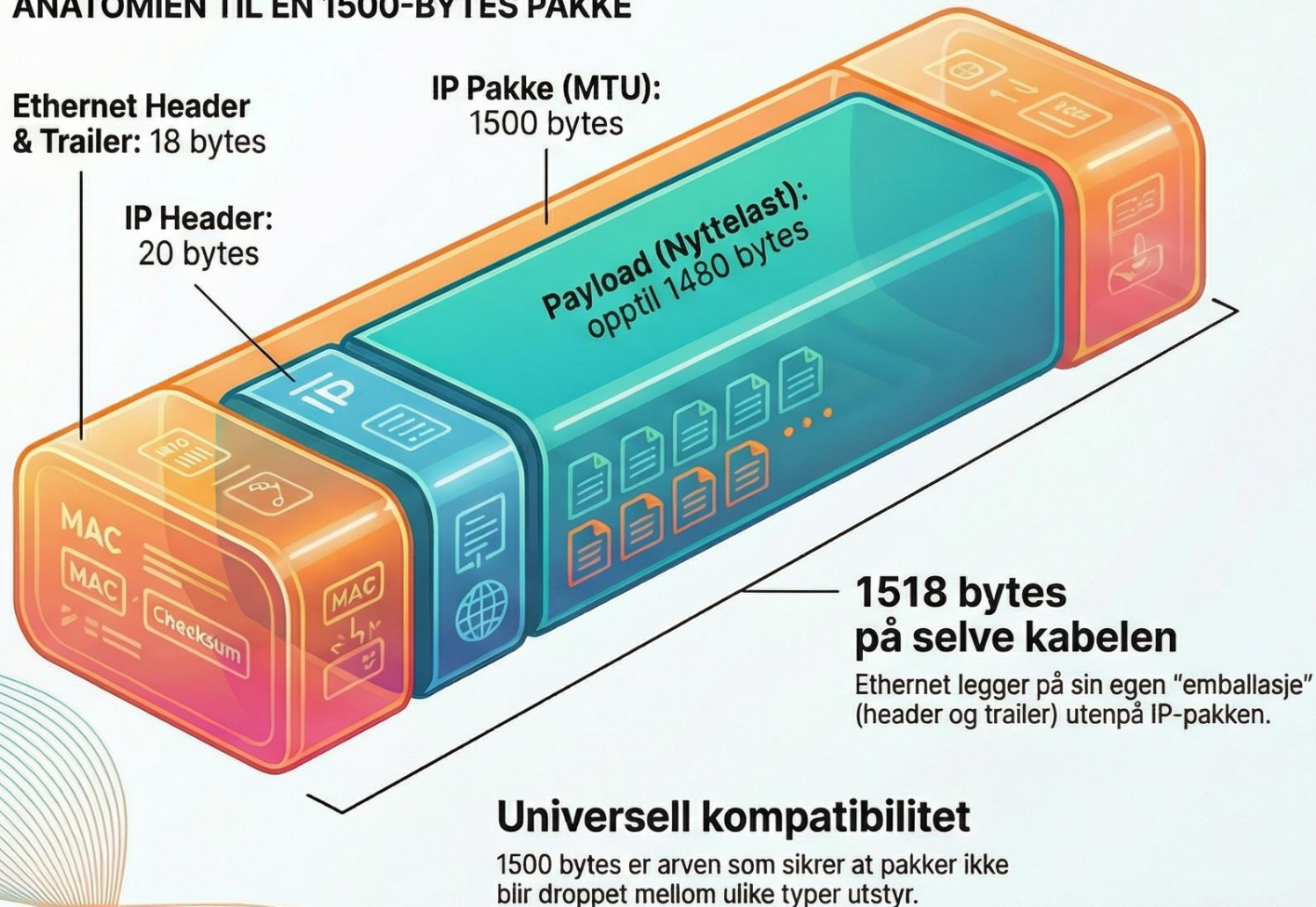


Transport Layer: 3-4

Forstå Standard MTU: Nettverkets "Fartsgrense"

MTU (Maximum Transmission Unit) bestemmer den maksimale størrelsen på en IP-pakke som kan sendes over et nettverksgrensesnitt. Standarden på 1500 bytes sikrer sømløs kommunikasjon over internett.

ANATOMIEN TIL EN 1500-BYTES PAKKE



VIKTIGE UNNTAK FRA REGELEN



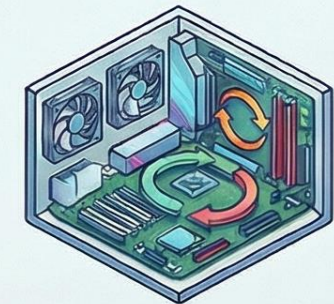
Jumbo Frames (9000 bytes)

Brukes i lukkede datasentre for høyere hastighet og lavere CPU-belastning.



PPPoE (1492 bytes)

Brukes ofte på fiber hjemme; krever 8 bytes ekstra plass til protokoll-innpakning.

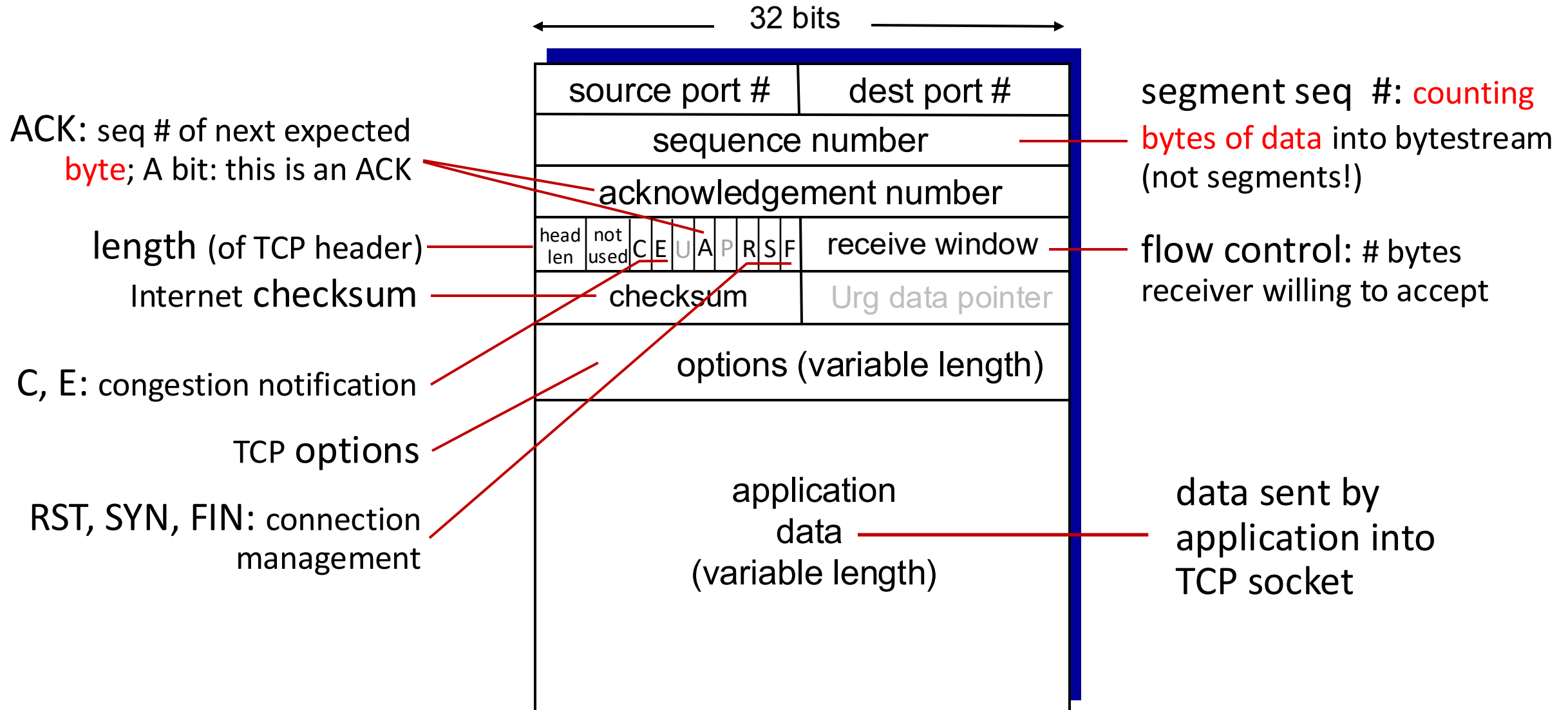


Loopback (65536 bytes)

Intern trafikk i datamaskinens minne har ingen fysiske begrensninger.

Transport Layer 3-5

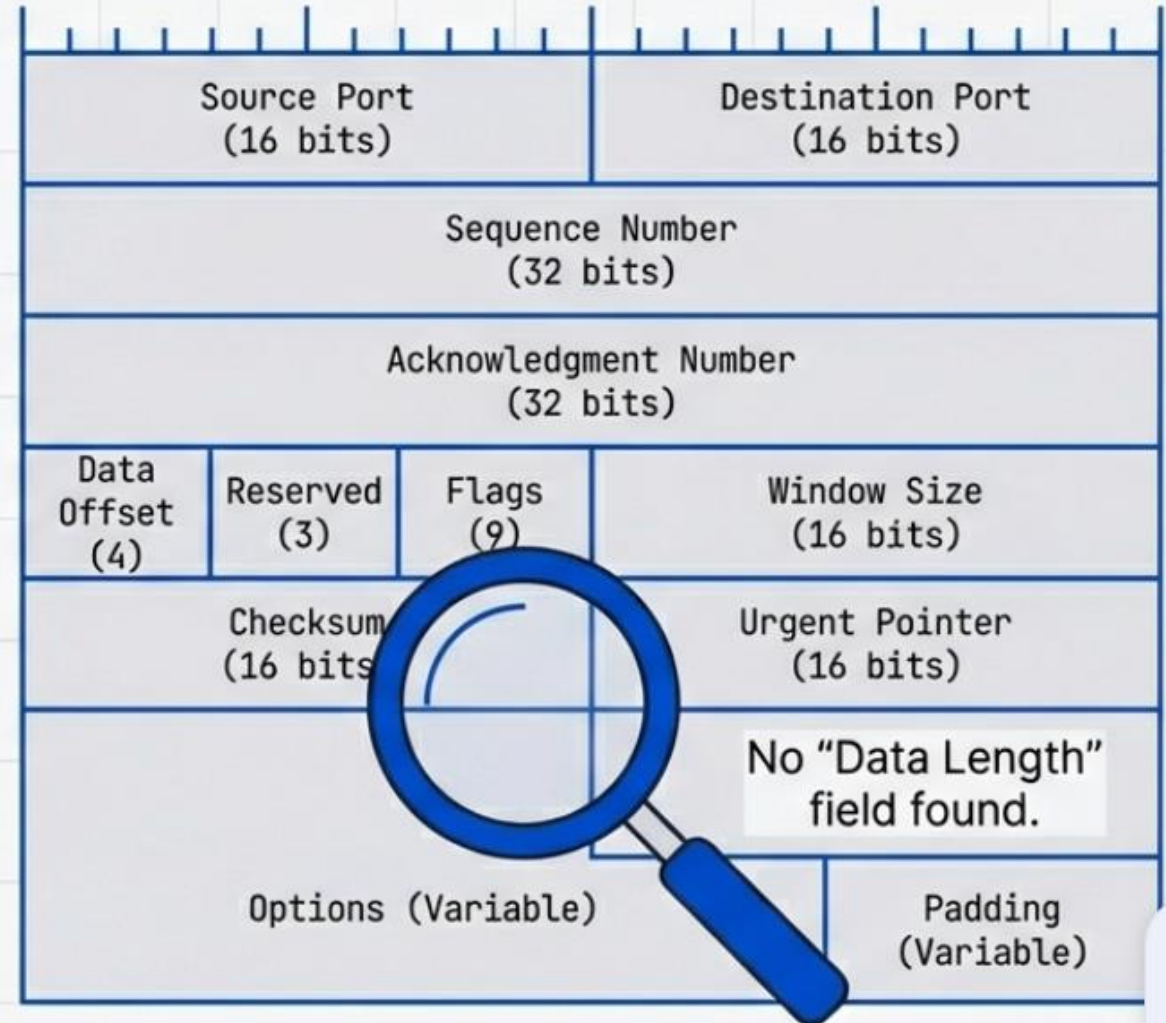
TCP segment structure



The Missing Field

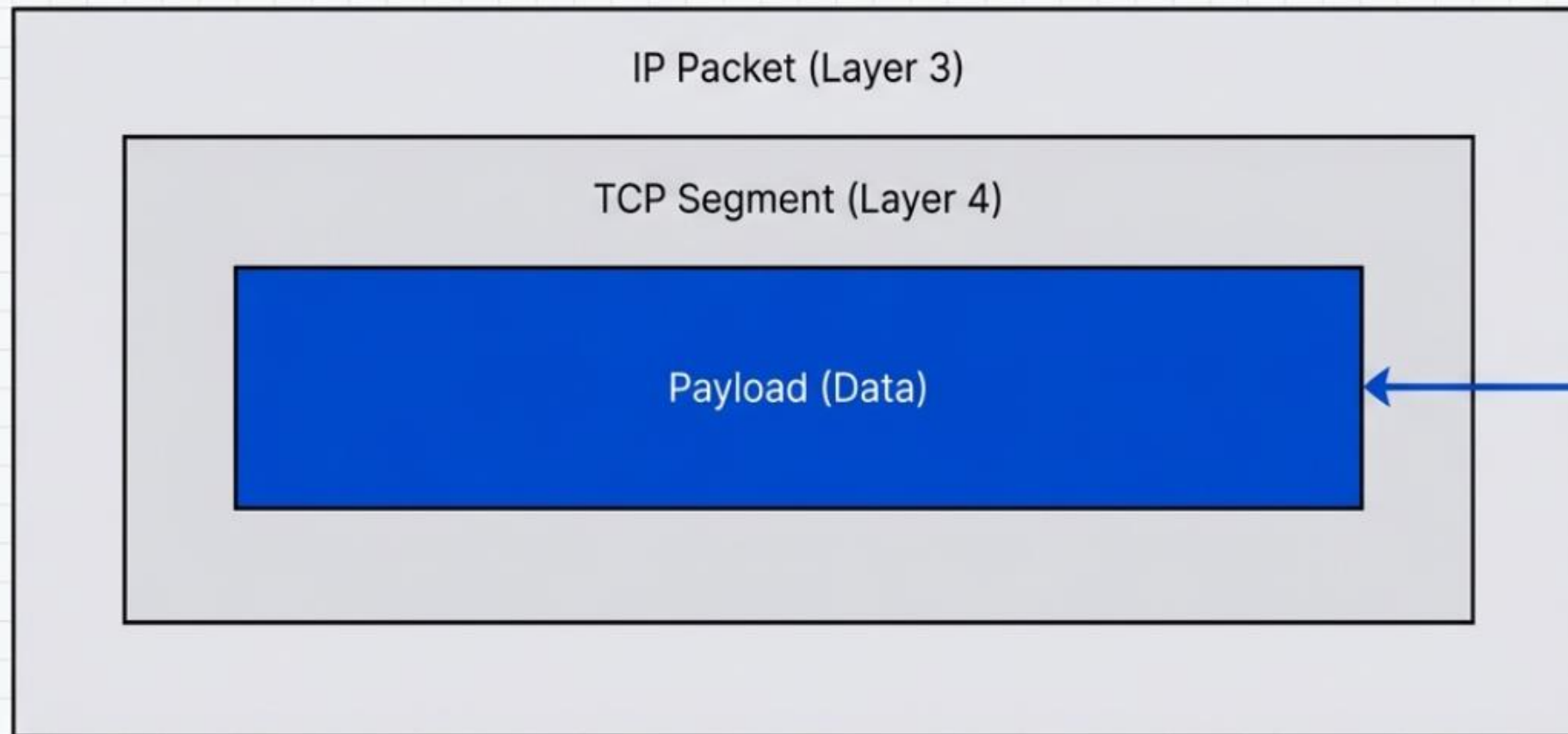
Unlike some protocols, TCP headers **do not carry a "Payload Length" value**. The receiver cannot simply read a number to know how much data it holds.

The Deduction: Instead, the receiver must calculate the payload size by looking at information provided by the the layer below (IP) and subtracting the overhead.



The Anatomy of Encapsulation

To understand how data size is calculated, we must visualize the nesting. The actual user data is wrapped inside a TCP Header, which is wrapped inside an IP Header.



The TCP protocol does not have a dedicated field that explicitly states "Here is the length of my data."

Deduction by Subtraction

The Operating System determines the size of the TCP payload using a specific formula:

$$\text{Payload} = \text{Total IP Length} - \text{IP Header} - \text{TCP Header}$$

TCP sequence numbers, ACKs

Sequence numbers:

- byte stream “number” of first byte in segment’s data
- (Altså IKKE pakke eller segment nr !!)

Acknowledgements:

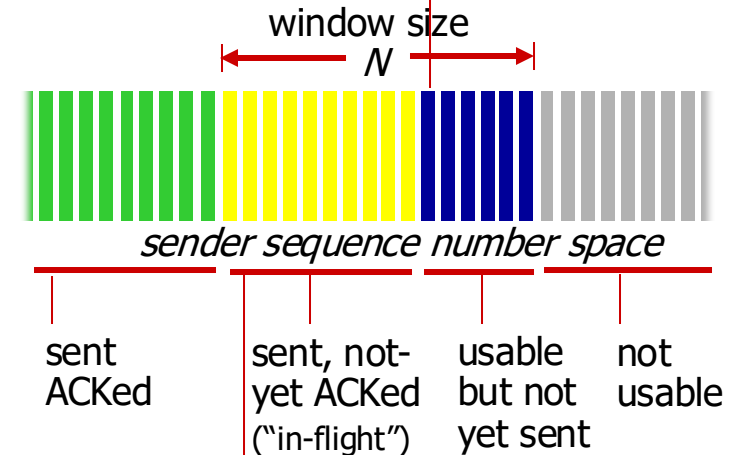
- seq # of next **byte** expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

- A: TCP spec doesn’t say, - up to implementor

outgoing segment from **sender**

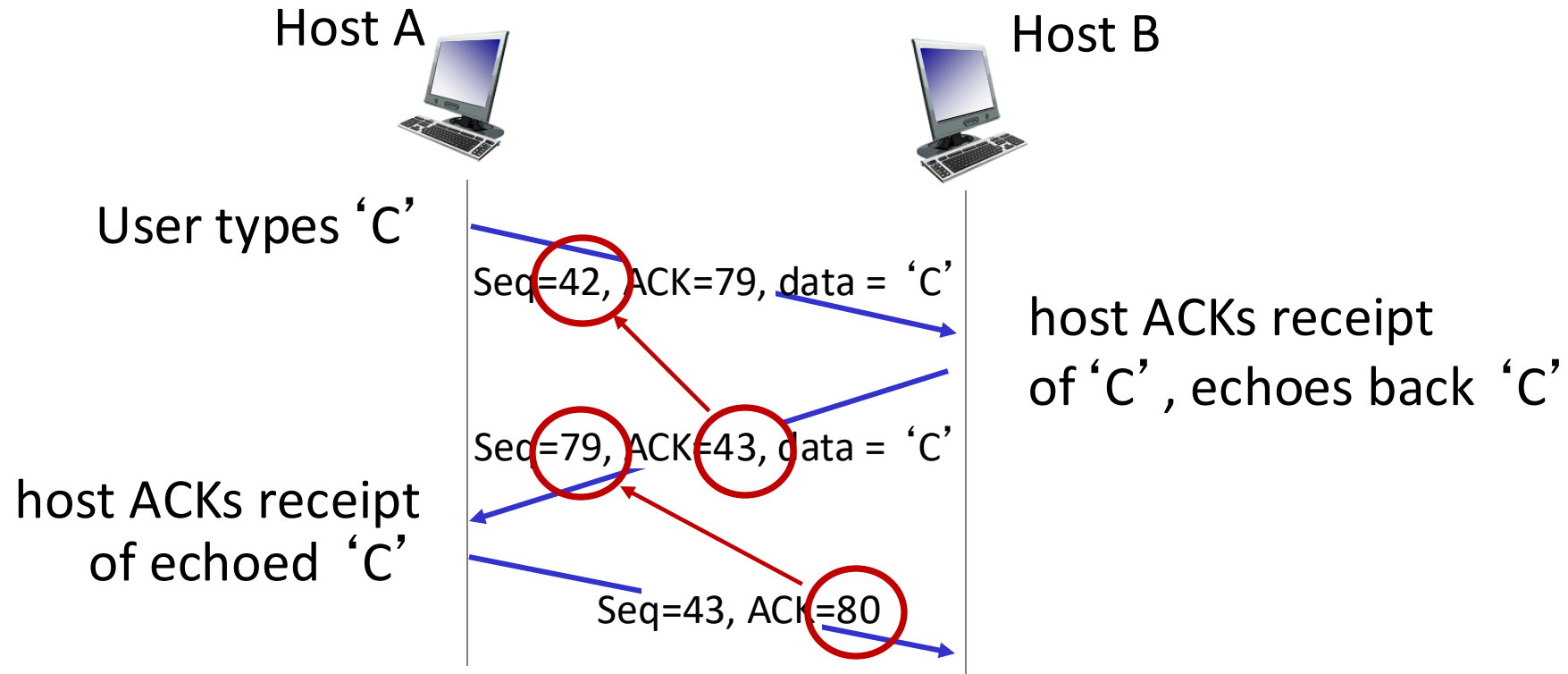
source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer



outgoing segment from **receiver**

source port #	dest port #
sequence number	
acknowledgement number	
	rwnd
checksum	urg pointer

TCP sequence numbers, ACKs



simple telnet scenario

TCP round trip time, timeout

Q: how to set TCP timeout value?

- longer than RTT, but RTT varies!
- *too short*: premature timeout, unnecessary retransmissions
- *too long*: slow reaction to segment loss

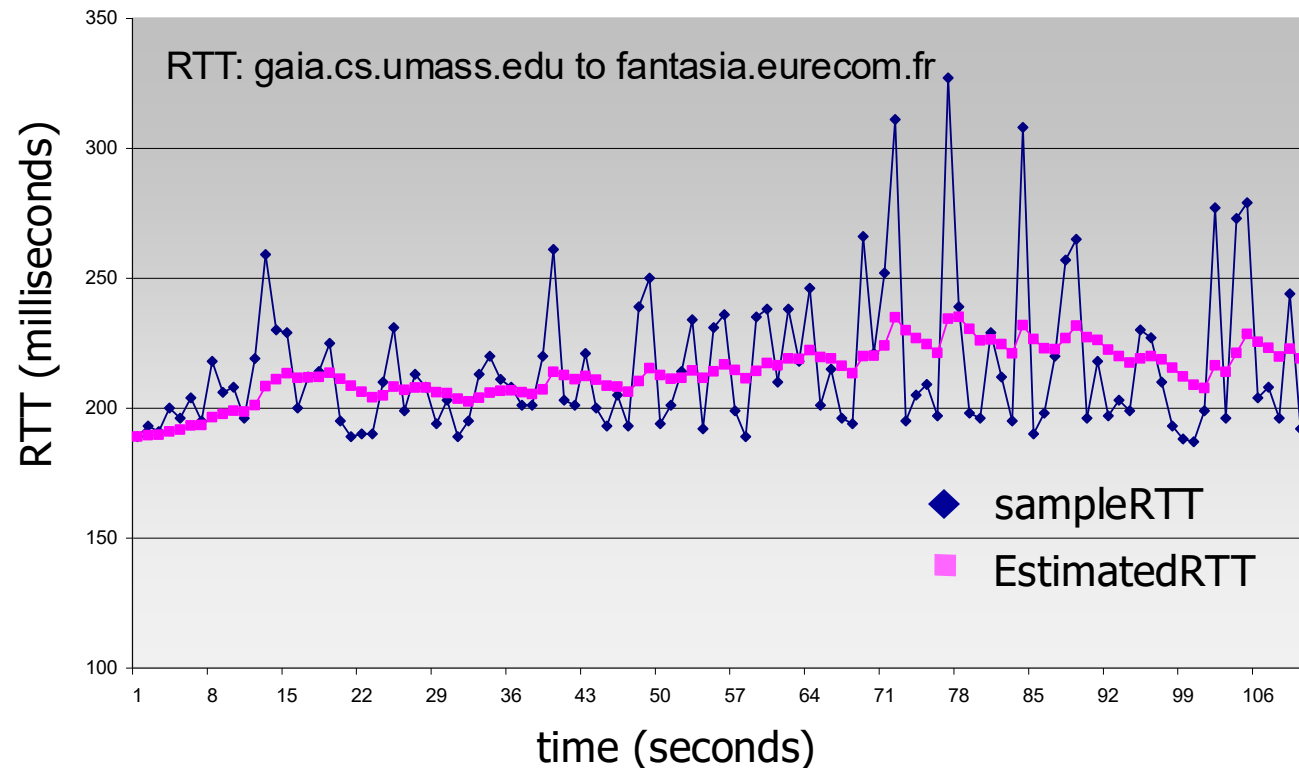
Q: how to estimate RTT?

- *SampleRTT*: measured time from segment transmission until ACK receipt
 - ignore retransmissions
- *SampleRTT* will vary, want estimated RTT “smoother”
 - average several *recent* measurements, not just current *SampleRTT*

TCP round trip time, timeout

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- exponential weighted moving average (EWMA)
- influence of past sample decreases exponentially fast
- typical value: $\alpha = 0.125$



TCP round trip time, timeout

- timeout interval: **EstimatedRTT** plus “safety margin”
 - large variation in **EstimatedRTT**: want a larger safety margin

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$$



↑
estimated RTT

↑
“safety margin”

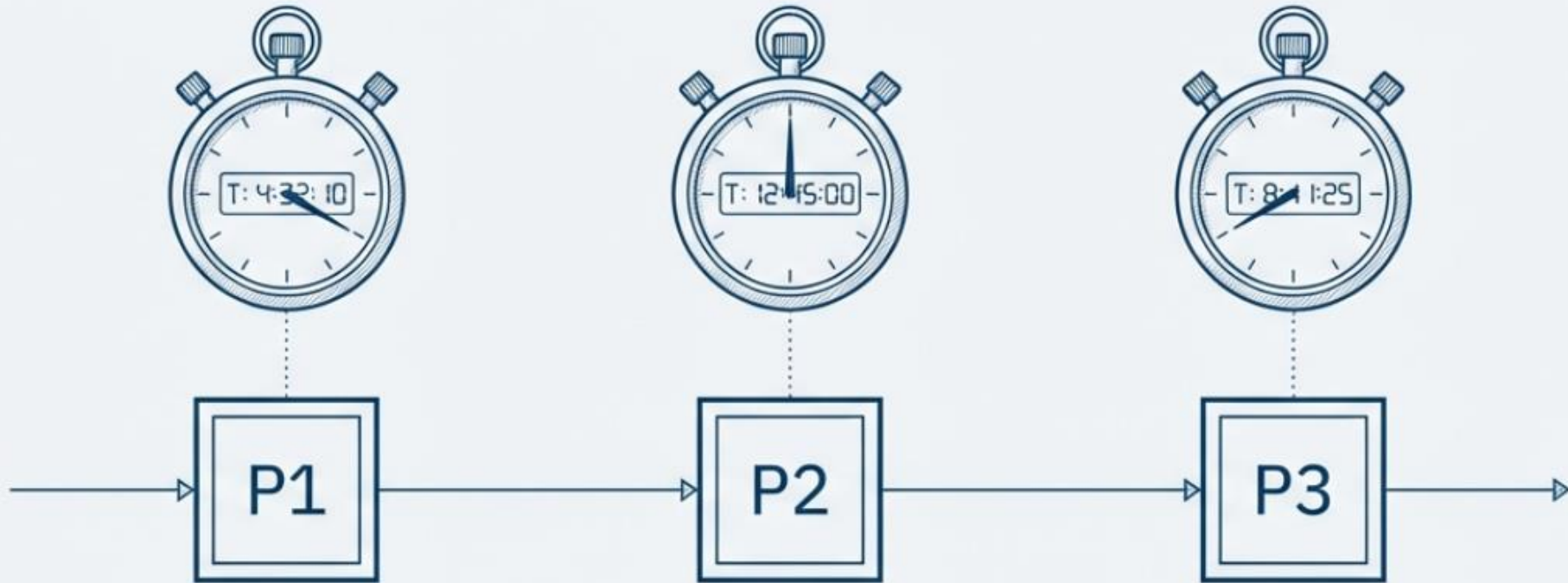
- **DevRTT**: **EWMA** of **SampleRTT** deviation from **EstimatedRTT**:

$$\text{DevRTT} = (1 - \beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

The Intuitive Model: One Packet, One Timer



Logically, reliable delivery implies individual tracking. If an OS sends Packet A, B, and C, it seems natural to spawn three separate timer objects to track the specific timeout window for each. In low-volume systems, this model works perfectly fine.

Why Intuition Fails at Scale

Low Volume



1 Connection. 1 Timer.
CPU Load: Negligible.

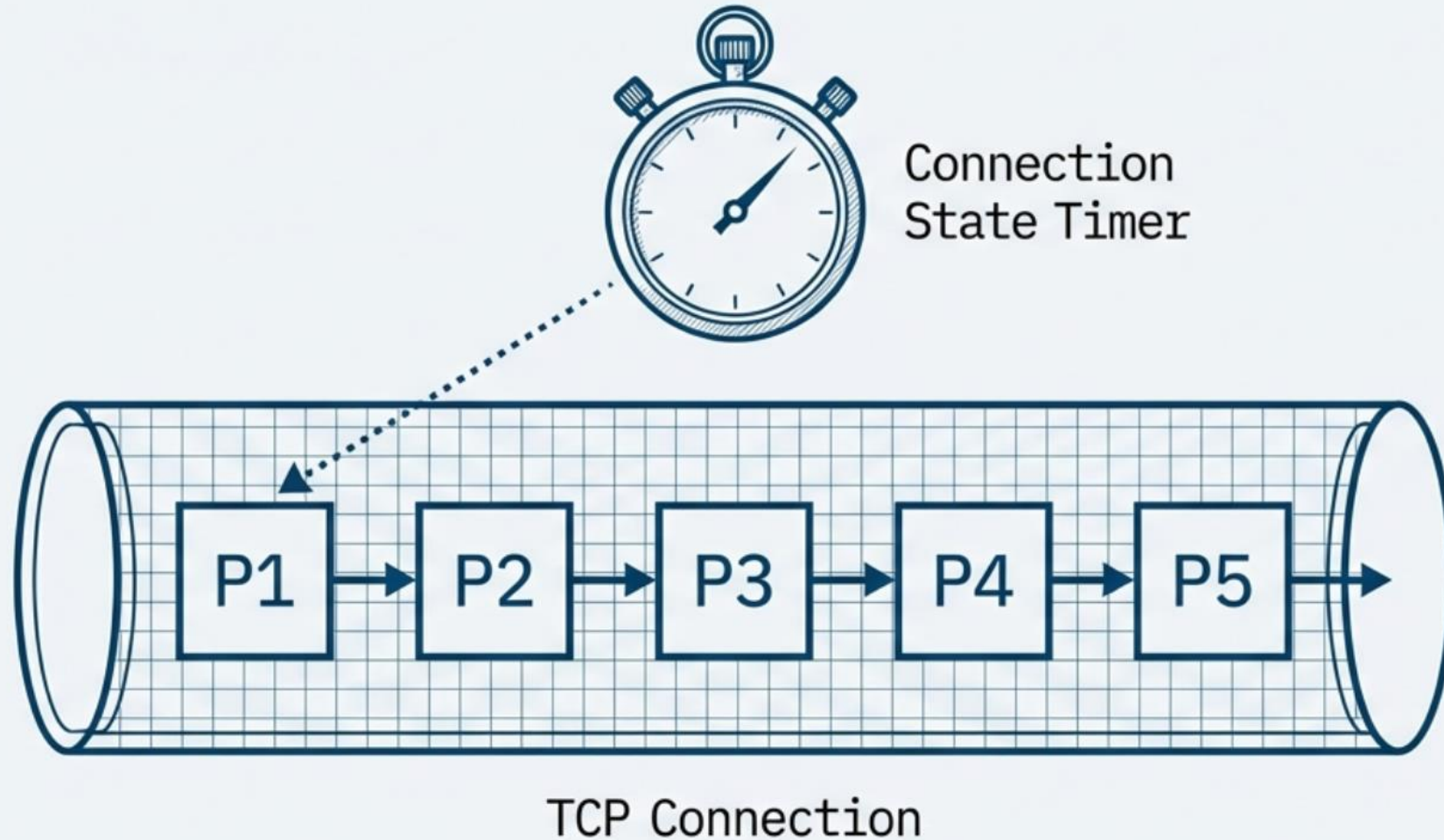
High Volume



**10,000 Connections x 1,000 Packets/sec
= Millions of Active Timers.**

If the operating system maintained a separate, active "timer object" for every TCP packet, the CPU would kneel. Context switching between millions of interrupt threads is computationally impossible. To survive, TCP must assume a constraint-driven de

The Reality: The Single Retransmission Timer



In standard TCP implementations, there is usually only ONE retransmission timer per TCP connection.

The OS does not track every packet actively; it tracks the connection's state relative to the oldest unacknowledged data.

TCP Sender (simplified)

event: data received from application

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
 - think of timer as for oldest unACKed segment
 - expiration interval: **TimeoutInterval**

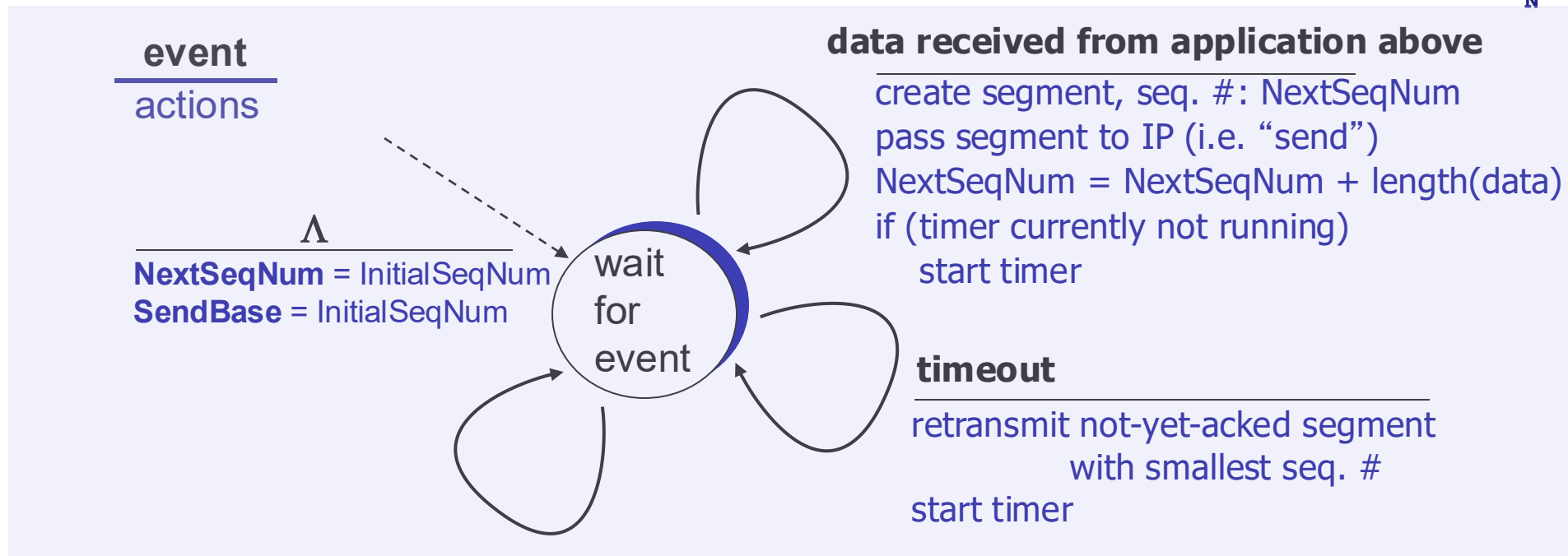
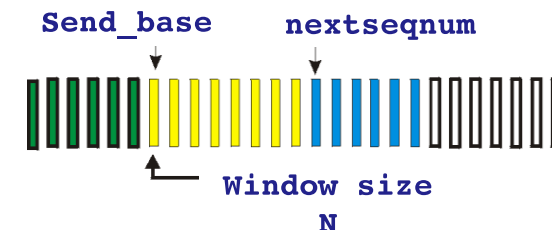
event: *timeout*

- retransmit segment that caused timeout
- restart timer

event: *ACK received*

- if ACK acknowledges previously unACKed segments
 - update what is known to be ACKed
 - start timer if there are still unACKed segments

TCP sender (simplified)

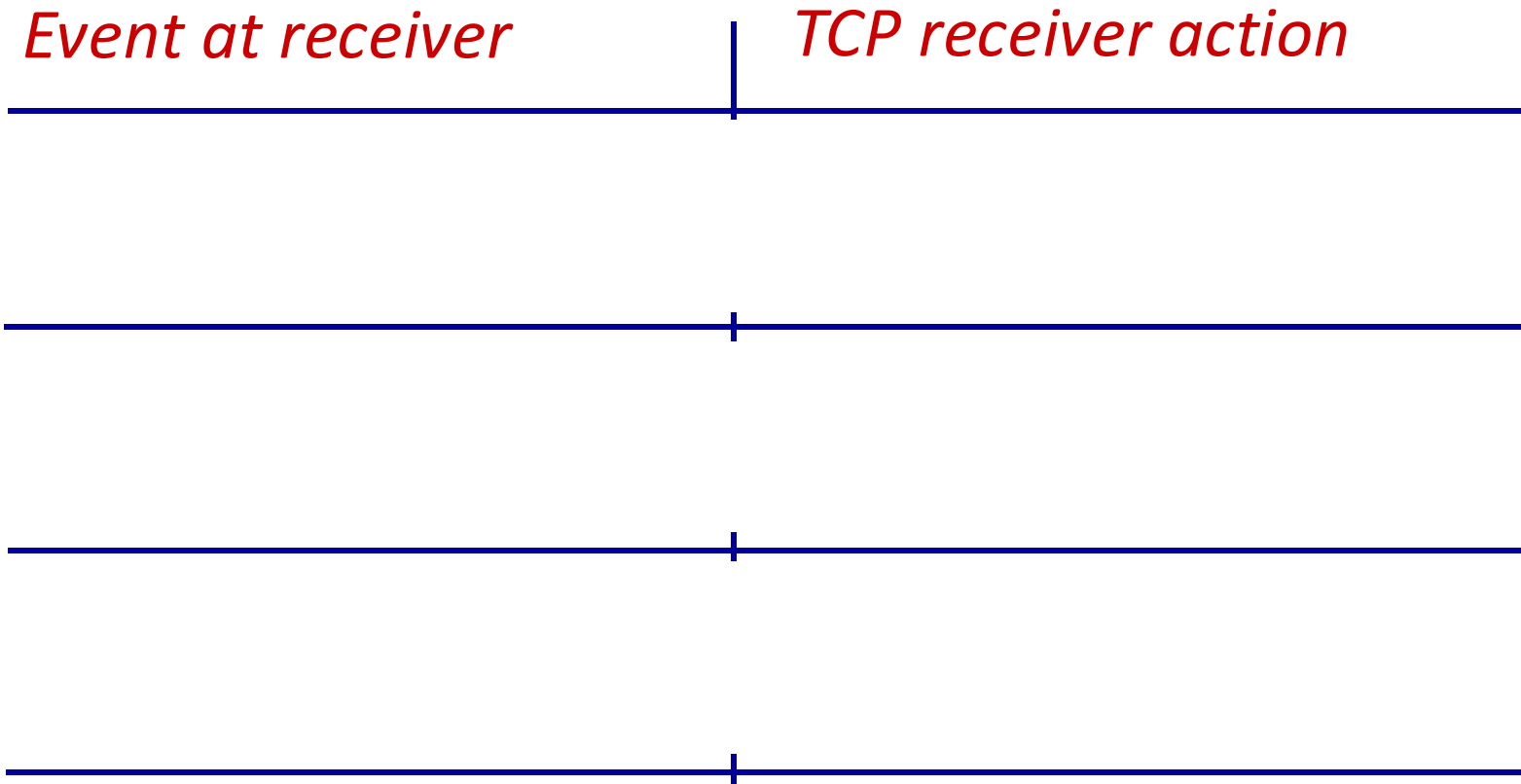


ACK received, with ACK field value y

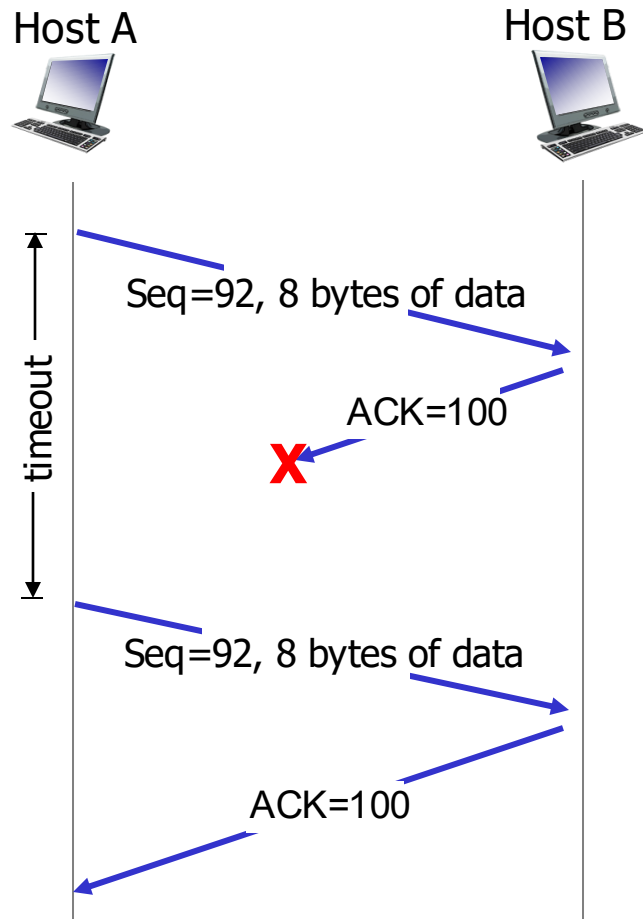
```

if (y > SendBase) {
  SendBase = y
  /* SendBase-1: last cumulatively ACKed byte */
  if (there are currently not-yet-acked segments)
    start timer
  else stop timer
}
  
```

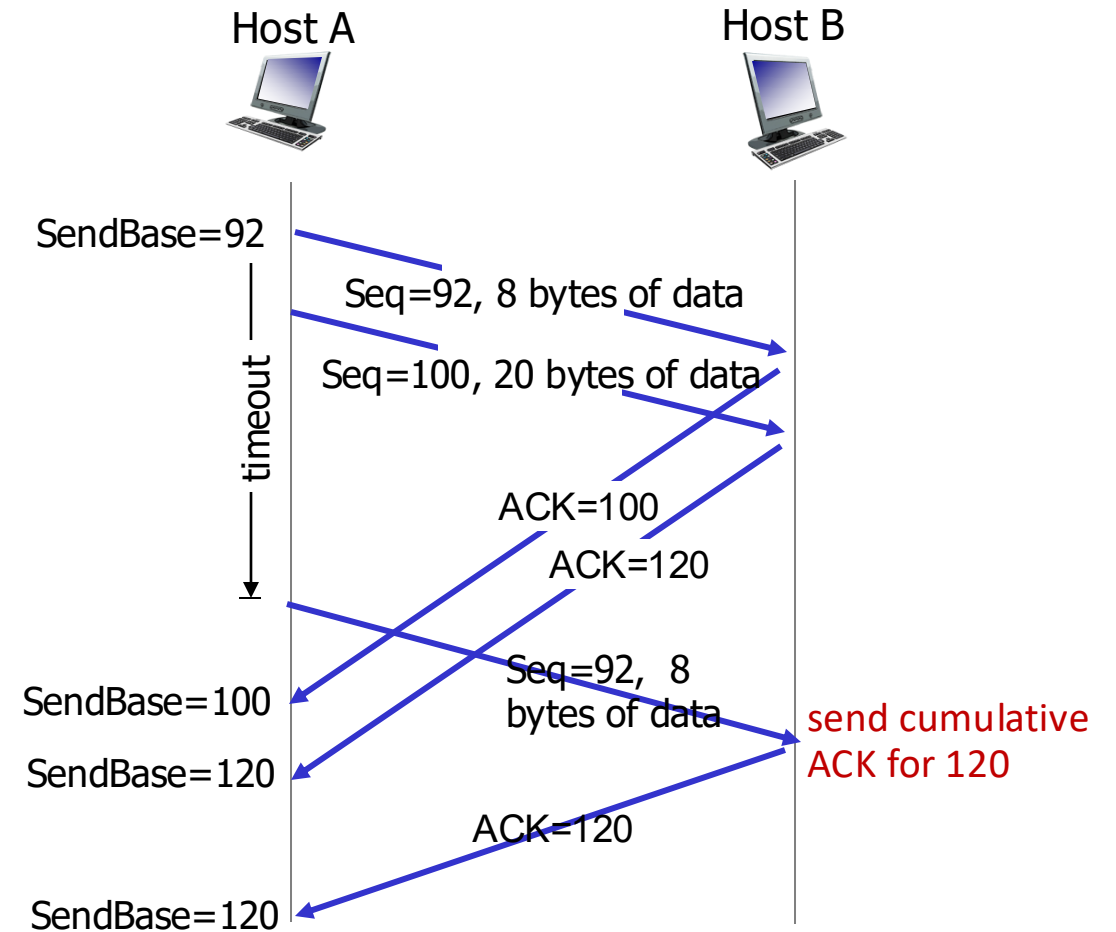
TCP Receiver: ACK generation [RFC 5681]



TCP: retransmission scenarios

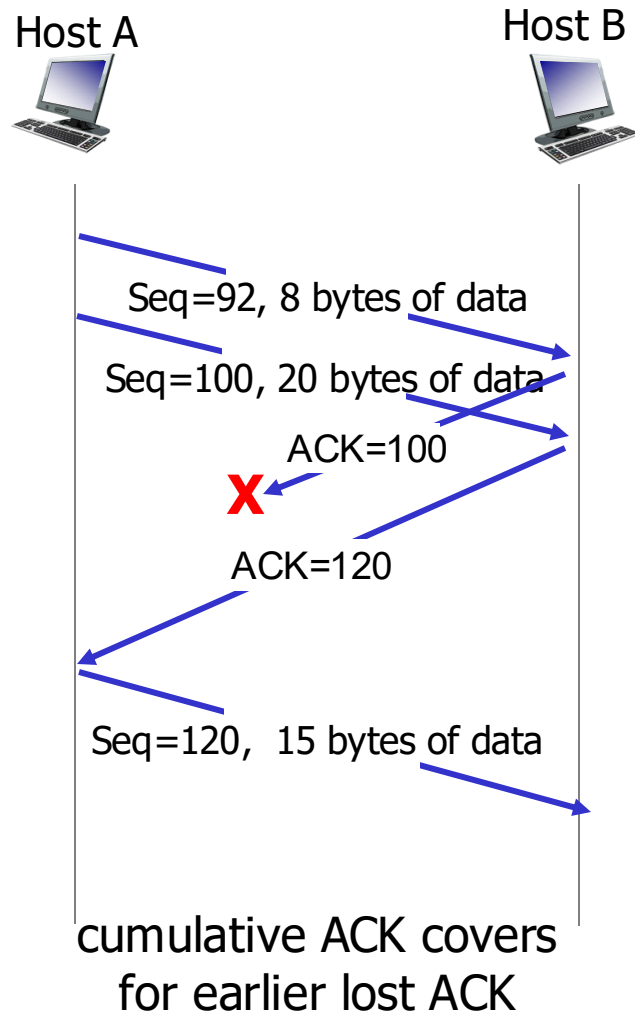


lost ACK scenario



premature timeout

TCP: retransmission scenarios



TCP fast retransmit

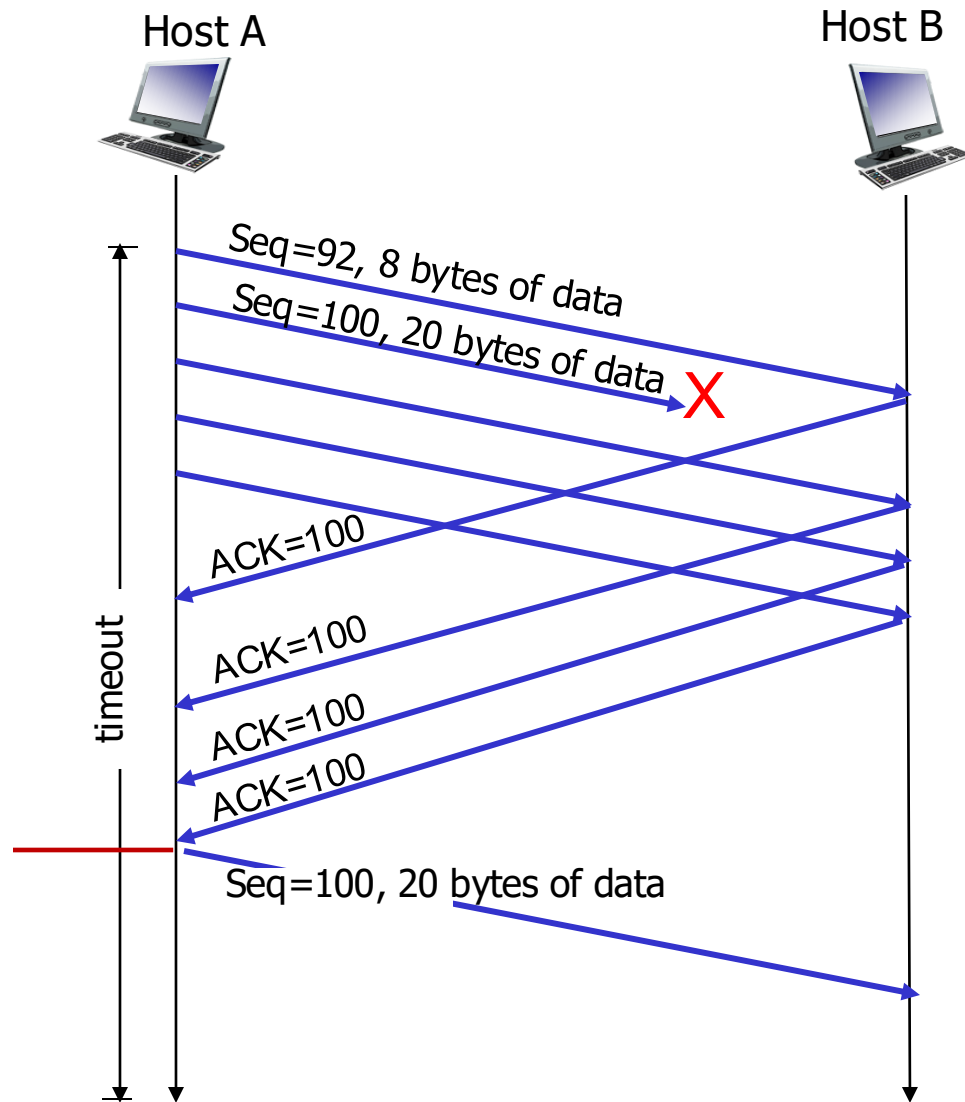
TCP fast retransmit

if sender receives 3 additional ACKs for same data (“triple duplicate ACKs”), resend unACKed segment with smallest seq #

- likely that unACKed segment lost, so don't wait for timeout



Receipt of three duplicate ACKs indicates 3 segments received after a missing segment – lost segment is likely. So retransmit!



TCP sender handling fast retransmit

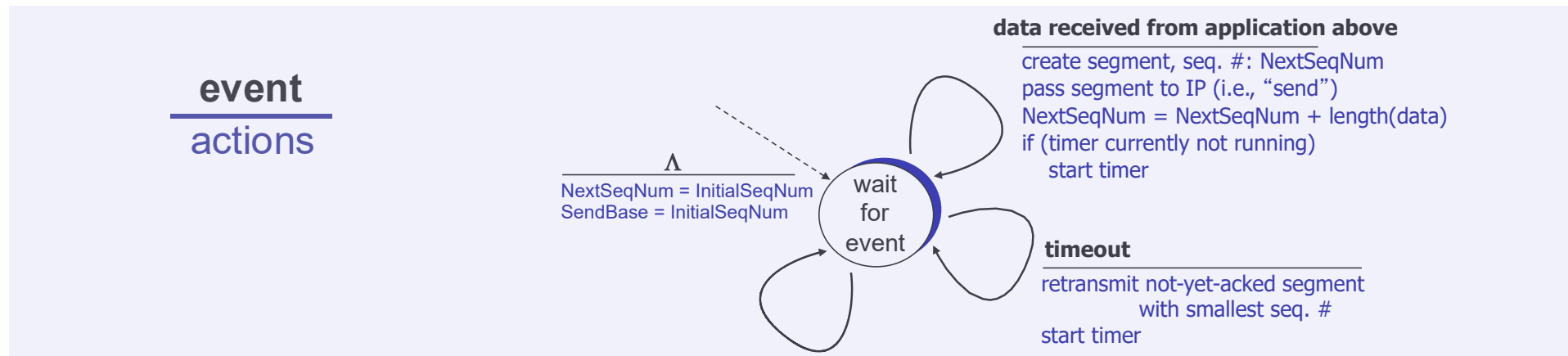
Hvorfor akkurat 3 dup ack før retransmit ?

3.2 Fast Retransmit/Fast Recovery

A TCP receiver SHOULD send an immediate duplicate ACK when an out- of-order segment arrives. The purpose of this ACK is to inform the sender that a segment was received out-of-order and which sequence number is expected. From the sender's perspective, duplicate ACKs can be caused by a number of network problems. First, they can be caused by dropped segments. In this case, all segments after the dropped segment will trigger duplicate ACKs. Second, duplicate ACKs can be caused by the re-ordering of data segments by the network (not a rare event along some network paths [Pax97]). Finally, duplicate ACKs can be caused by replication of ACK or data segments by the network. In addition, a TCP receiver SHOULD send an immediate ACK when the incoming segment fills in all or part of a gap in the sequence space. This will generate more timely information for a sender recovering from a loss through a retransmission timeout, a fast retransmit, or an experimental loss recovery algorithm, such as NewReno [FH98].

We noted earlier that many subtle issues arise when a timeout/retransmit mechanism is implemented in an actual protocol such as TCP. The procedures above, which have evolved as a result of more than 20 years of experience with TCP timers, should convince you that this is indeed the case!

TCP sender handling fast retransmit



event: ACK received, with ACK field value of y

```

if (y > SendBase) {
    SendBase = y
    if (there are currently not-yet-acknowledged segments)
        start timer
}
else {
    increment count of dup ACKs received for y
    if (count of dup ACKs received for y = 3) {
        resend segment with sequence number y
    }
}

```

a duplicate ACK for
already ACKed
segment

fast retransmit

